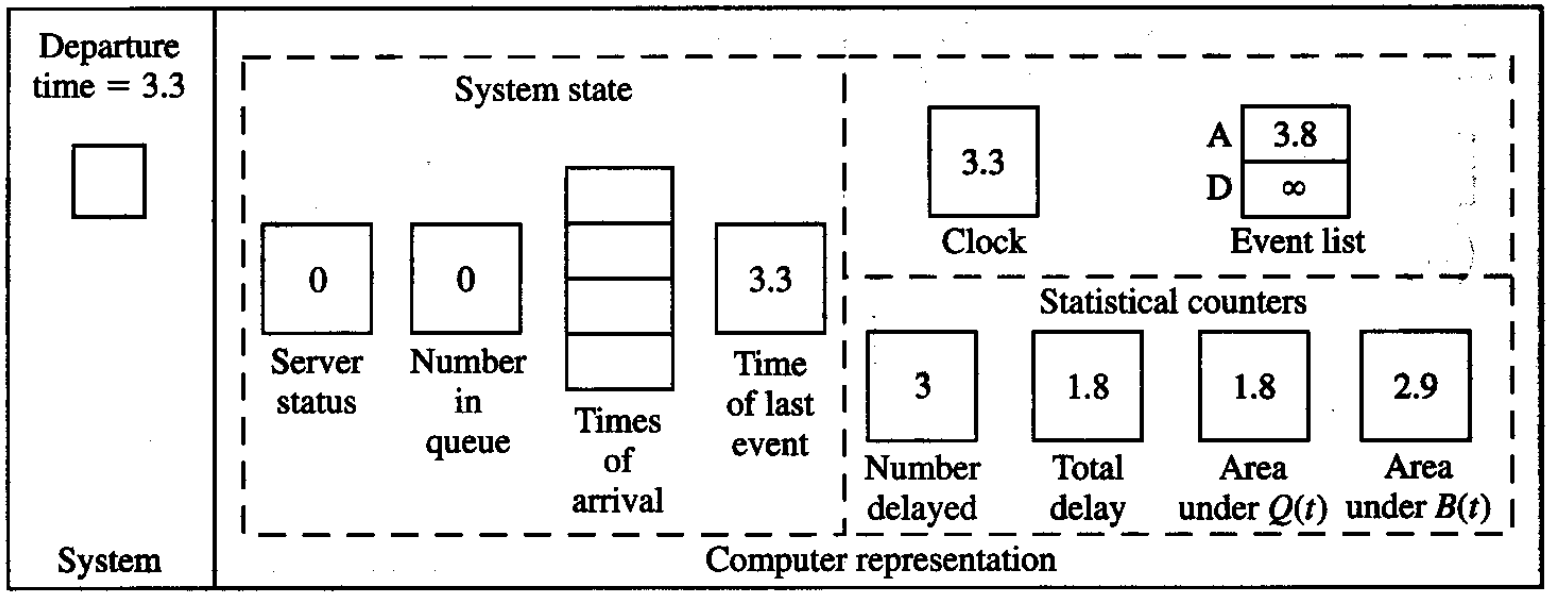
“Hand” DES Example

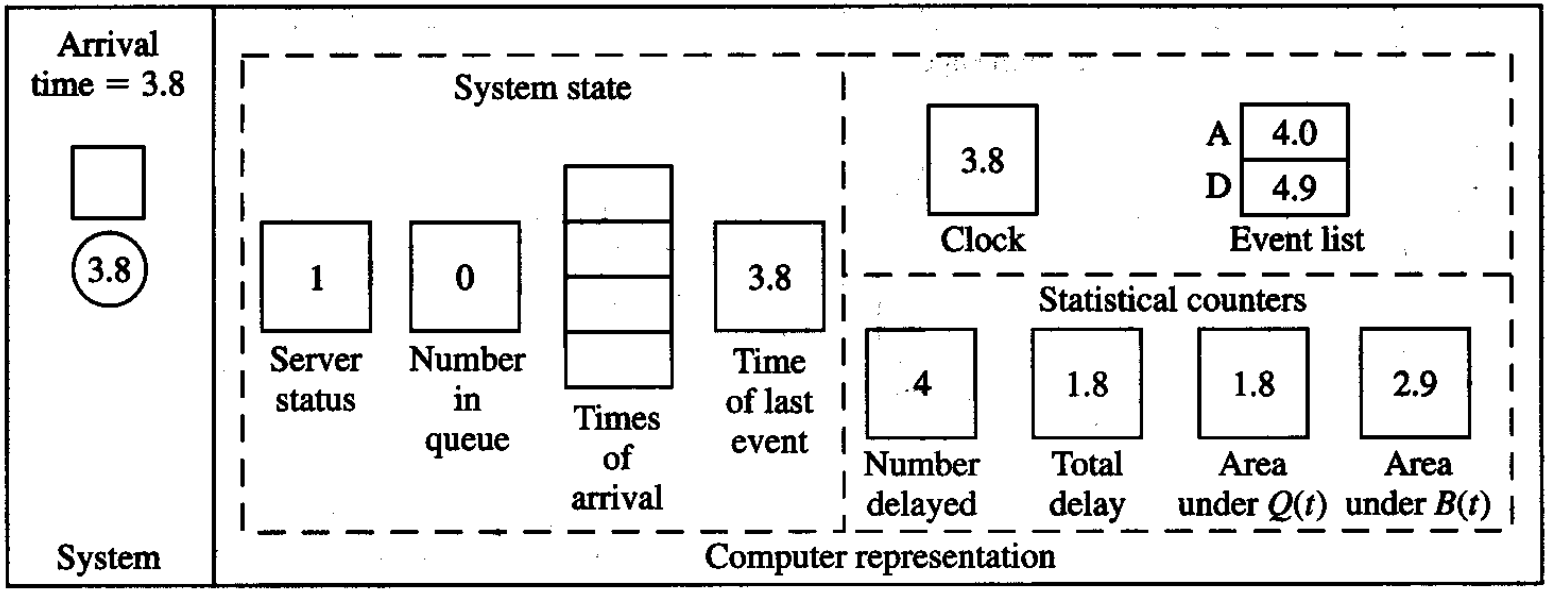


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, 0.2, 1.6, 0.2, 1.4, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, 1.1, 3.7, 0.6, …

- ΔT = 3,3 – 3,1 = 0,2
- Area Under B(t) += 0,2 -> 2,7 + 0,2 = 2,9
- Area under Q(t) += 0*0,2 -> 1,8 + 0 = 1,8
- Total delay += 0

“Hand” DES Example

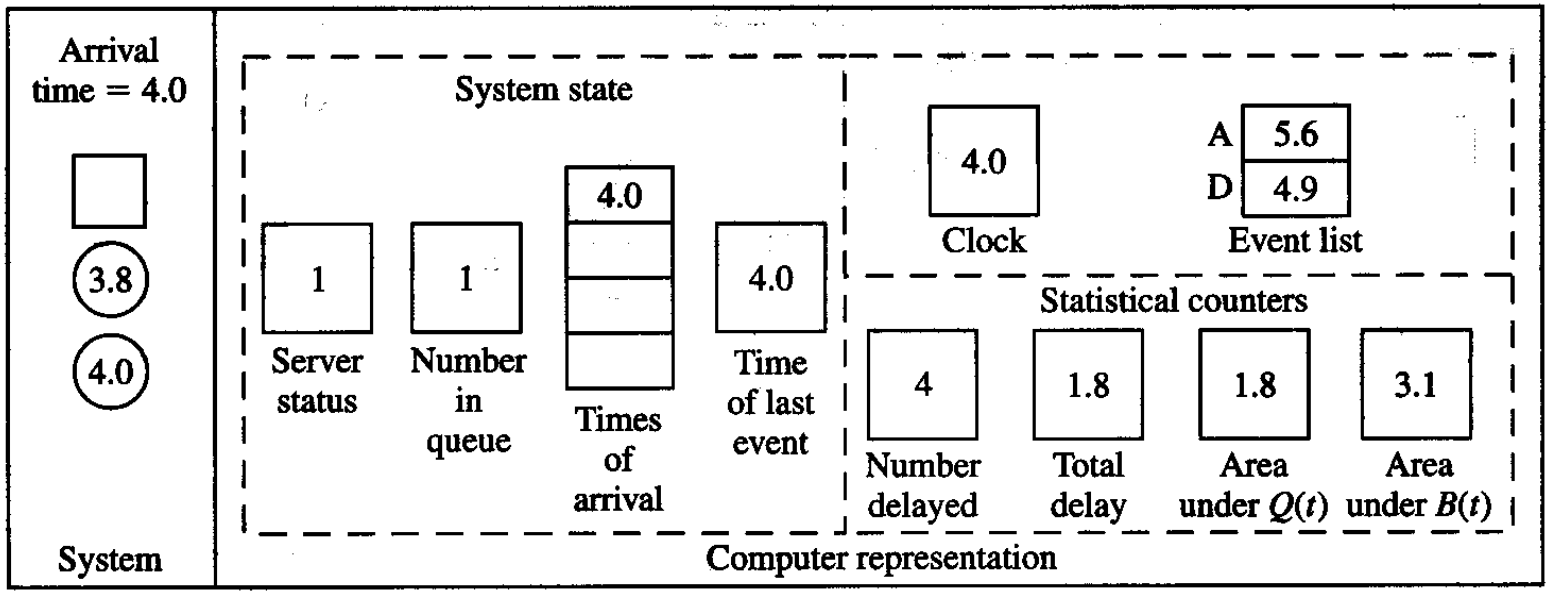


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, 1.6, 0.2, 1.4, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, 3.7, 0.6, …

- ΔT = 3,8 – 3,3 = 0,5
- Area Under B(t) += 0 -> 2,9 + 0 = 2,9
- Area under Q(t) += 0*0,5 -> 1,8 + 0 = 1,8
- Total delay += 0

“Hand” DES Example

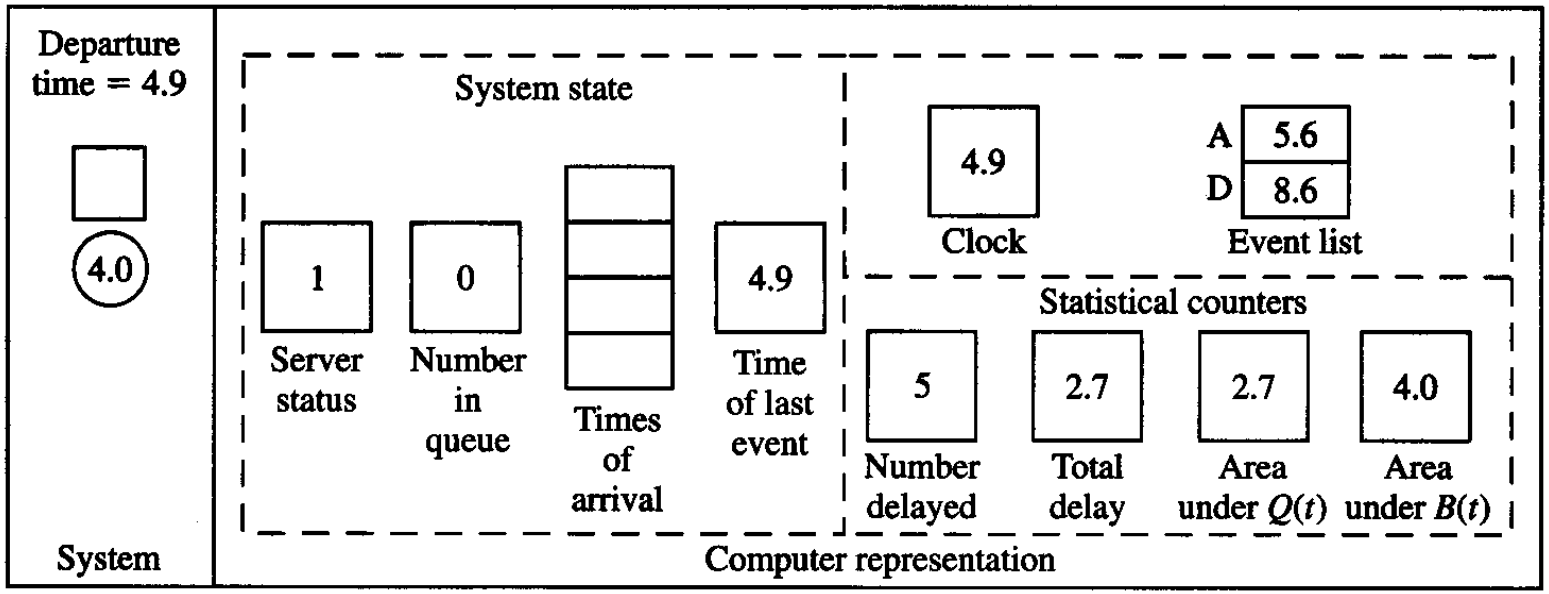


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, ~~1.6~~, 0.2, 1.4, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, 3.7, 0.6, …

- ΔT = 4,0 – 3,8 = 0,2
- Area Under B(t) += 0,2 -> 2,9 + 0,2 = 3,1
- Area under Q(t) += 0*0,2 -> 1,8 + 0 = 1,8
- Total delay += 0

“Hand” DES Example

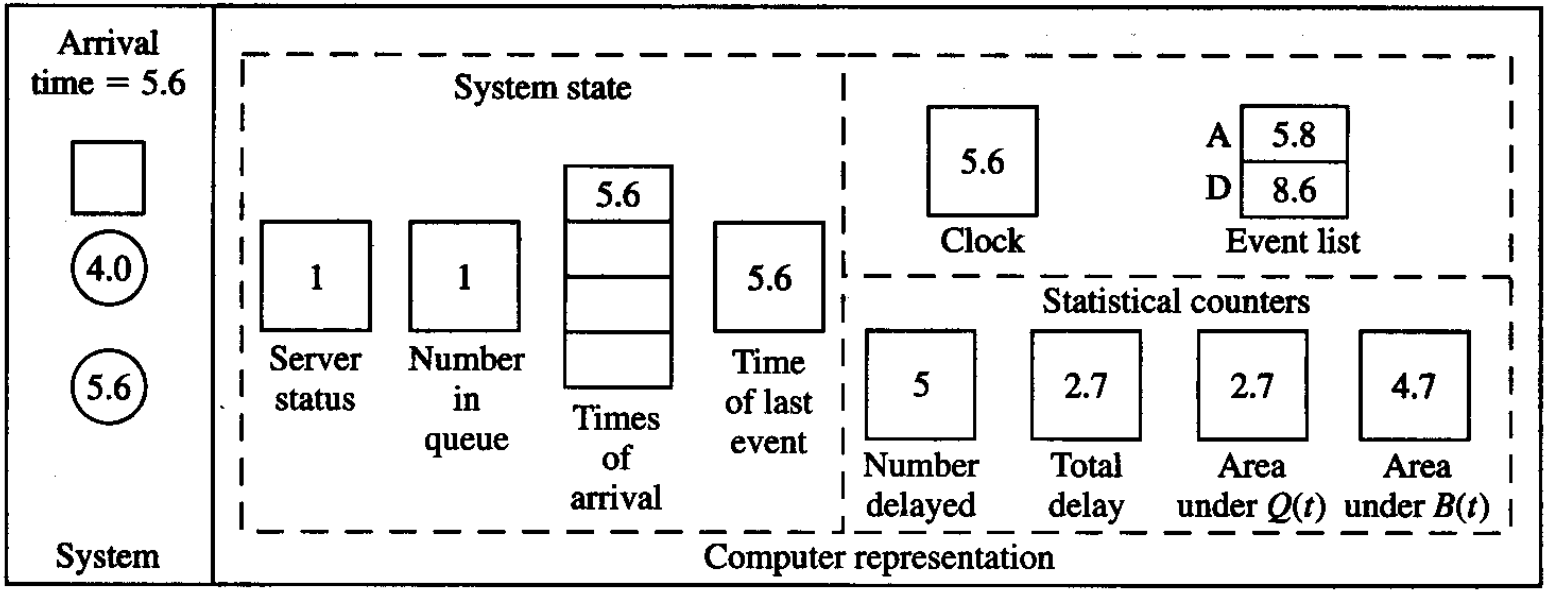


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, ~~1.6~~, 0.2, 1.4, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, ~~3.7~~, 0.6, …

- ΔT = 4,9 – 4,0 = 0,9
- Area Under B(t) += 0,9 -> 3,1 + 0,9 = 4,0
- Area under Q(t) += 1*0,9 -> 1,8 + 0,9 = 2,7
- Total delay += 4,9 – 4,0 -> 1,8 + (4,9 – 4,0) = 2,7

“Hand” DES Example

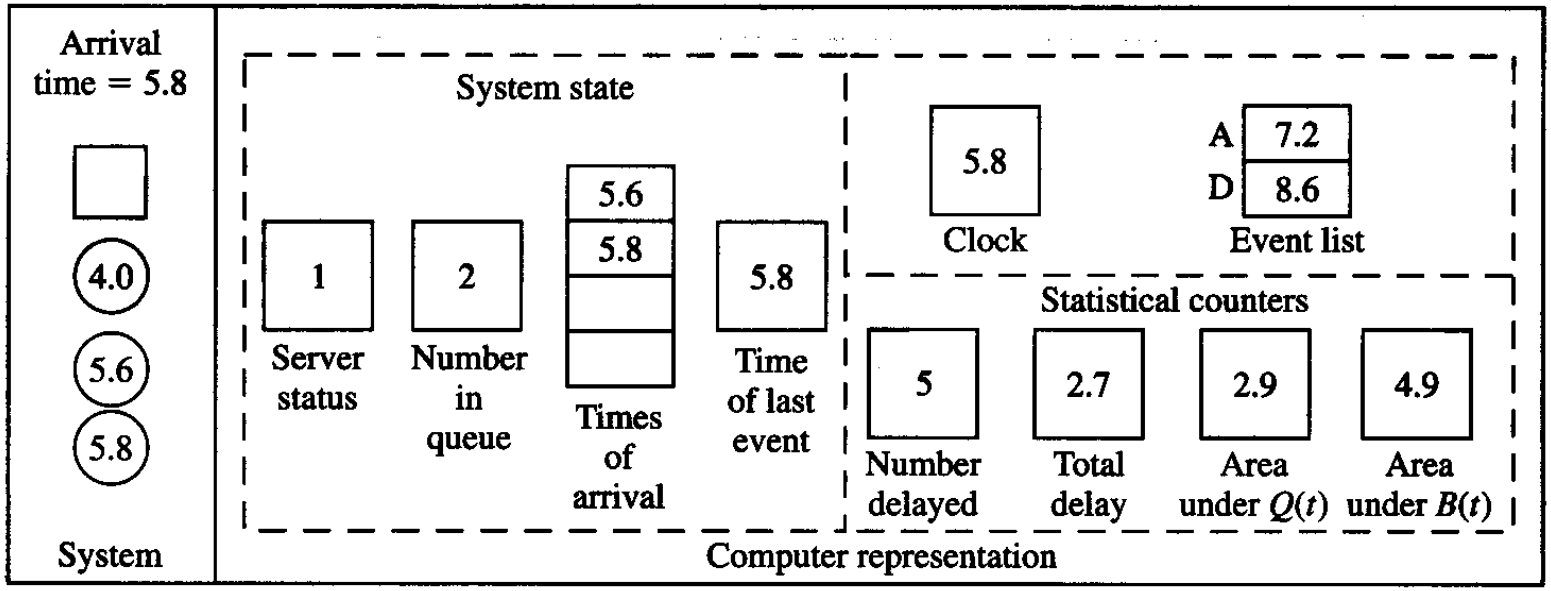


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, ~~1.6~~, ~~0.2~~, 1.4, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, ~~3.7~~, 0.6, …

- ΔT = 5,6 – 4,9 = 0,7
- Area Under B(t) += 0,7 -> 4,0 + 0,7 = 4,7
- Area under Q(t) += 0*0,7 -> 2,7
- Total delay += 0

“Hand” DES Example

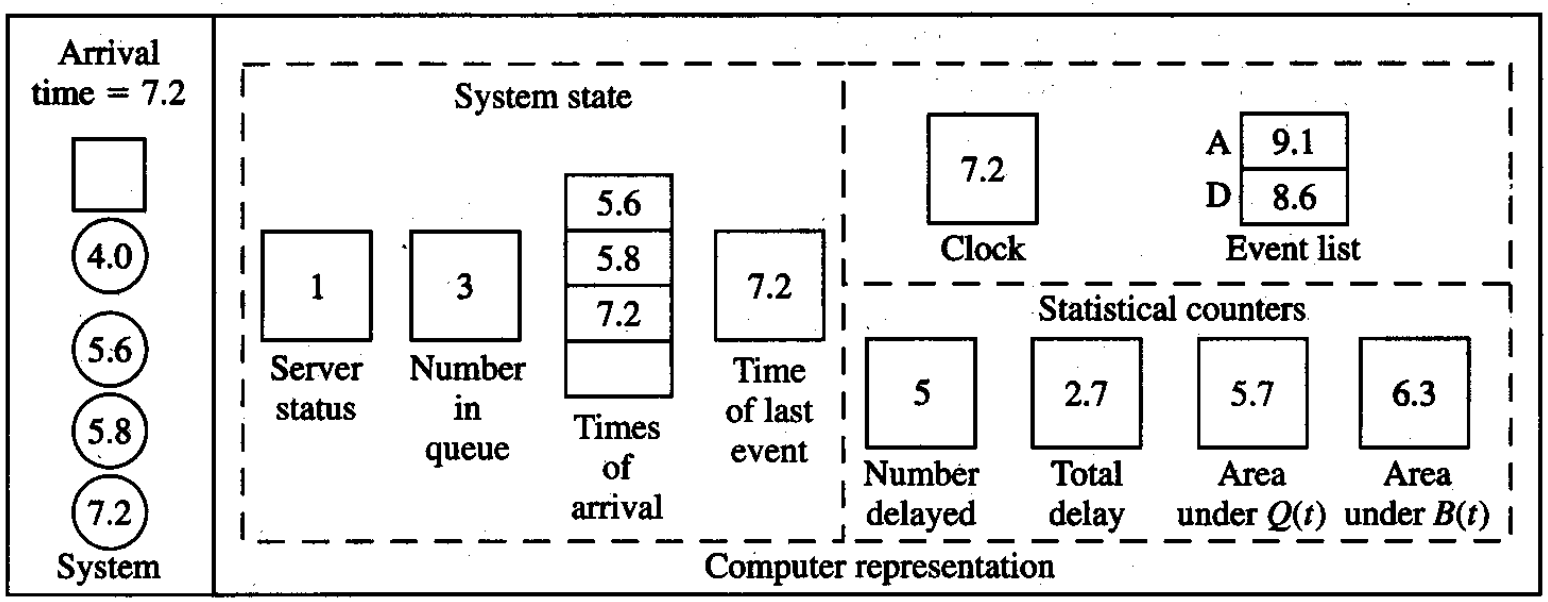


Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, ~~1.6~~, ~~0.2~~, ~~1.4~~, 1.9, …

Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, ~~3.7~~, 0.6, …

- ΔT = 5,8 – 5,6 = 0,2
- Area Under B(t) += 0,2 -> 4,7 + 0,2 = 4,9
- Area under Q(t) += 1*0,2 -> 2,7 + 0,2 = 2,9
- Total delay += 0

“Hand” DES Example



Interarrival times: ~~0.4~~, ~~1.2~~, ~~0.5~~, ~~1.7~~, ~~0.2~~, ~~1.6~~, ~~0.2~~, ~~1.4~~, ~~1.9~~, …

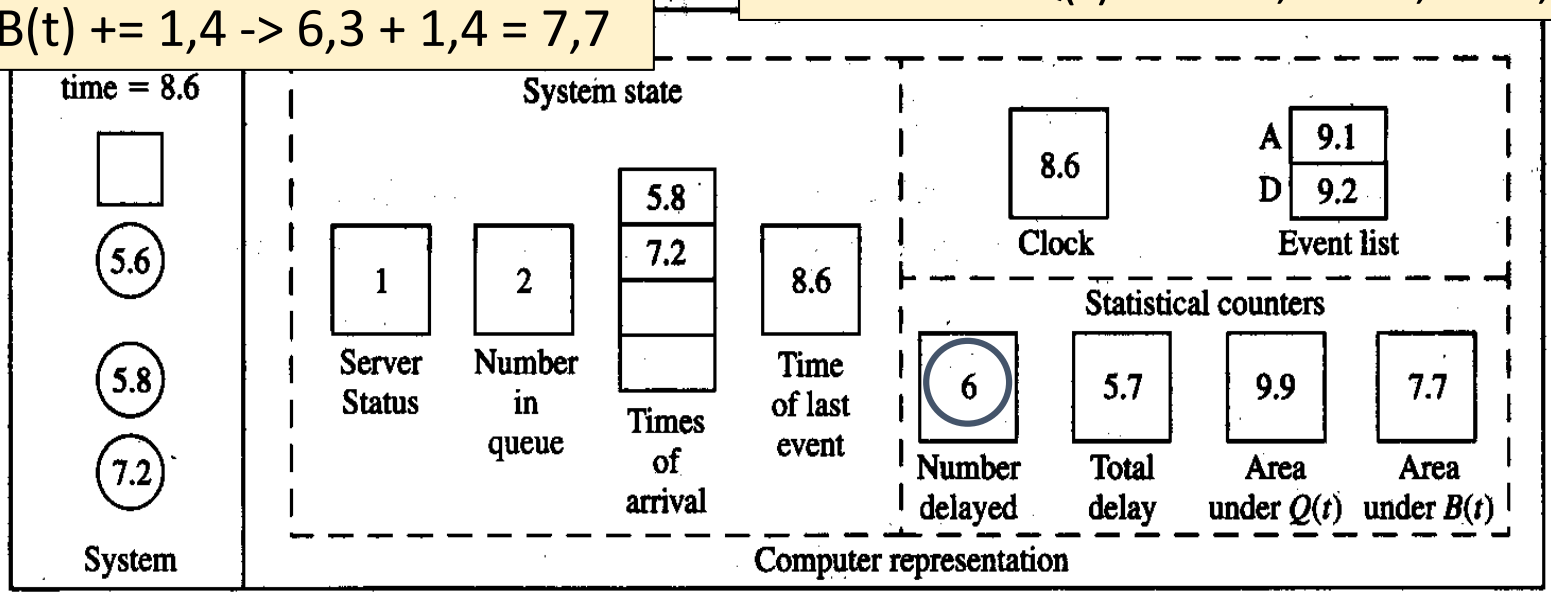
Service times: ~~2.0~~, ~~0.7~~, ~~0.2~~, ~~1.1~~, ~~3.7~~, 0.6, …

- ΔT = 7,2 – 5,8 = 1,4
- Area Under B(t) += 1,4 -> 4,9 + 1,4 = 6,3
- Area under Q(t) += 2*1,4 -> 2,9 + 2,8 = 5,7
- Total delay += 0

“Hand” DES Example

ΔT = 8,6 – 7,2 = 1,4
Area Under B(t) += 1,4 -> 6,3 + 1,4 = 7,7

Area under Q(t) += 3*1,4 -> 5,7 + 4,2 = 9,9



Interarrival times: 0.4, 1.2, 0.5, 1.7, 0.2, 1.6, 0.2, 1.4, 1.9, …

Service times: 2.0, 0.7, 0.2, 1.1, 3.7, 0.6, …

Final output performance measures:

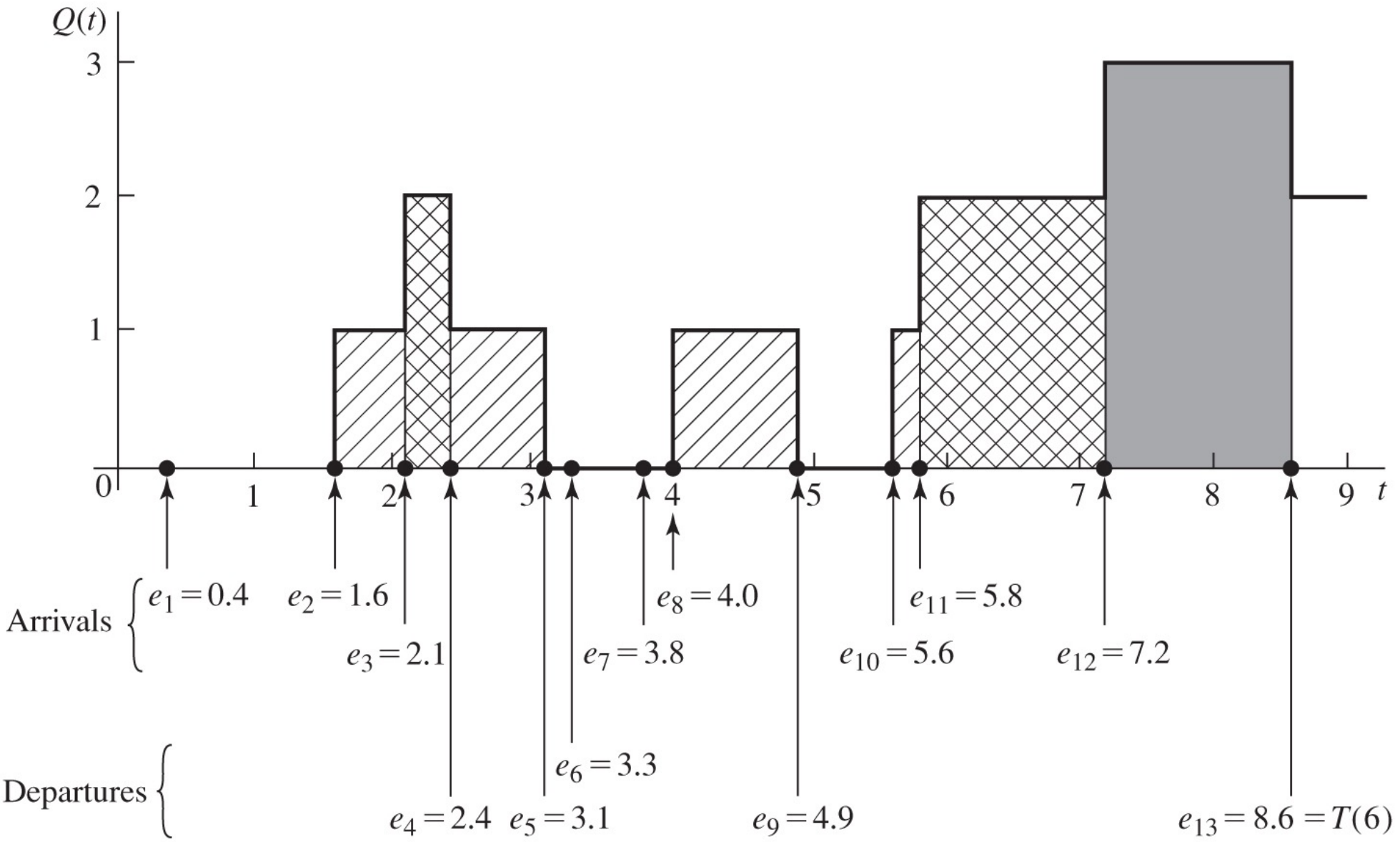
Average delay in queue = 5.7/6 = 0.95 min./cust.

Time-average number in queue = 9.9/8.6 = 1.15 cust.

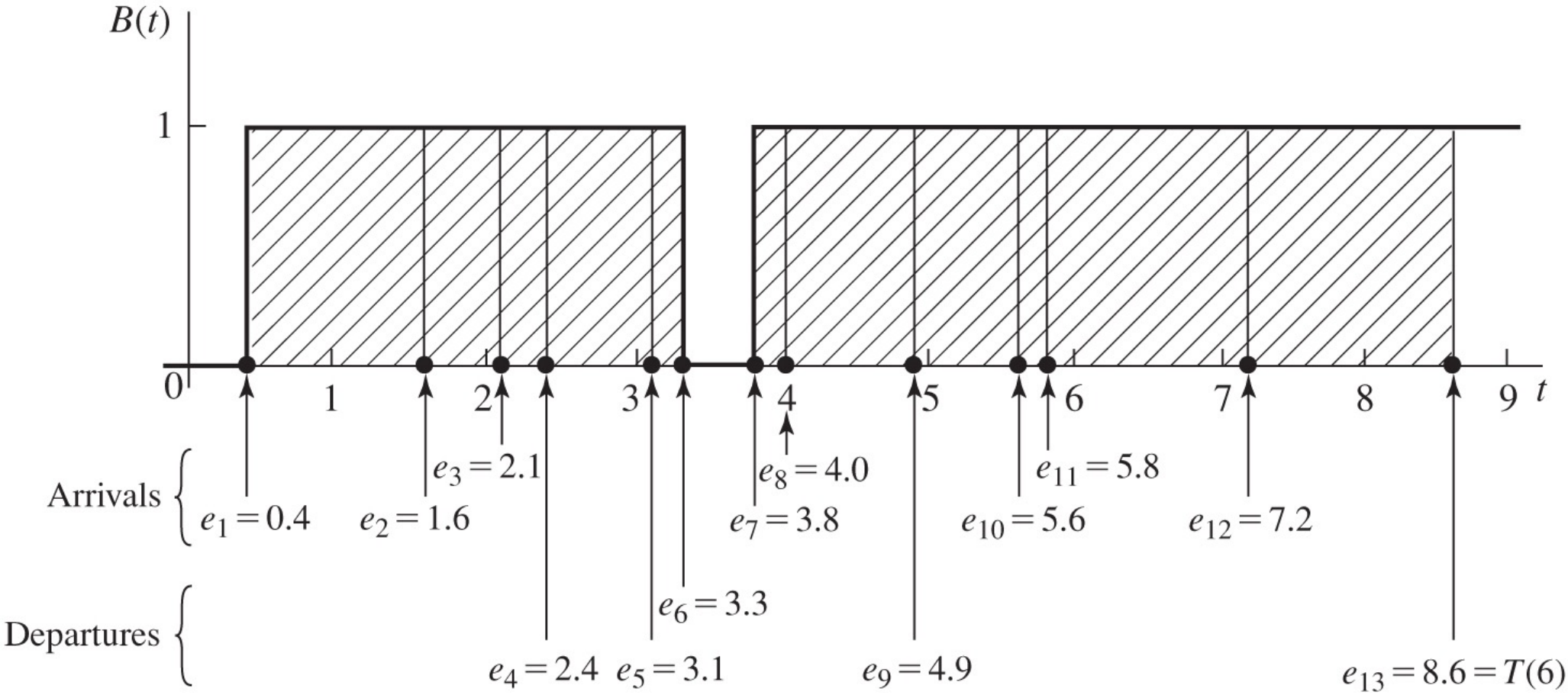
Server utilization = 7.7/8.6 = 0.90 (dimensionless)

Total delay += 8,6, – 5,6 -> 2,7 + (8,6 – 5,6) = 5,7

Area under Q(t)



Area under B(t)



DES Example – *C implementation*

```
/* External definitions for single-server queueing system, fixed run length. */

#include <stdio.h>
#include <math.h>
#include "lcgrand.h"  /* Header file for random-number generator. */

#define Q_LIMIT 100  /* Limit on queue length. */
#define BUSY      1  /* Mnemonics for server's being busy */
#define IDLE      0  /* and idle. */

int   next_event_type, num_custs_delayed, num_events, num_in_q, server_status;
float area_num_in_q, area_server_status, mean_interarrival, mean_service,
      sim_time, time_arrival[Q_LIMIT + 1], time_end, time_last_event,
      time_next_event[4], total_of_delays;
FILE  *infile, *outfile;

void  initialize(void);
void  timing(void);
void  arrive(void);
void  depart(void);
void  report(void);
void  update_time_avg_stats(void);
float expon(float mean);
```

DES Example – *C implementation*

```
main()  /* Main function. */
{
    /* Open input and output files. */

    infile  = fopen("mm1alt.in",  "r");
    outfile = fopen("mm1alt.out", "w");

    /* Specify the number of events for the timing function. */

    num_events = 3;

    /* Read input parameters. */

    fscanf(infile, "%f %f %f", &mean_interarrival, &mean_service, &time_end);

    /* Write report heading and input parameters. */

    fprintf(outfile, "Single-server queueing system with fixed run");
    fprintf(outfile, " length\n\n");
    fprintf(outfile, "Mean interarrival time%11.3f minutes\n\n",
            mean_interarrival);
    fprintf(outfile, "Mean service time%16.3f minutes\n\n", mean_service);
    fprintf(outfile, "Length of the simulation%9.3f minutes\n\n", time_end);
```

DES Example – *C implementation*

```
    initialize();

    /* Run the simulation until it terminates after an end-simulation event
       (type 3) occurs. */

    do
    {
        /* Determine the next event. */

        timing();

        /* Update time-average statistical accumulators. */

        update_time_avg_stats();

        /* Invoke the appropriate event function. */

        switch (next_event_type)
        {
            case 1:
                arrive();
                break;
            case 2:
                depart();
                break;
            case 3:
                report();
                break;
        }

    /* If the event just executed was not the end-simulation event (type 3),
       continue simulating.  Otherwise, end the simulation. */

    } while (next_event_type != 3);

    fclose(infile);
    fclose(outfile);

    return 0;
}
```

DES Example – *C implementation*

```
void initialize(void)  /* Initialization function. */
{
    /* Initialize the simulation clock. */

    sim_time = 0.0;

    /* Initialize the state variables. */

    server_status   = IDLE;
    num_in_q        = 0;
    time_last_event = 0.0;

    /* Initialize the statistical counters. */

    num_custs_delayed  = 0;
    total_of_delays    = 0.0;
    area_num_in_q      = 0.0;
    area_server_status = 0.0;

    /* Initialize event list.  Since no customers are present, the departure
       (service completion) event is eliminated from consideration.  The end-
       simulation event (type 3) is scheduled for time time_end. */

    time_next_event[1] = sim_time + expon(mean_interarrival);
    time_next_event[2] = 1.0e+30;
    time_next_event[3] = time_end;
}
```

DES Example – *C implementation*

```
void timing(void)  /* Timing function. */
{
    int   i;
    float min_time_next_event = 1.0e+29;

    next_event_type = 0;

    /* Determine the event type of the next event to occur. */

    for (i = 1; i <= num_events; ++i)
            if (time_next_event[i] < min_time_next_event) {
            min_time_next_event = time_next_event[i];
            next_event_type     = i;
        }

    /* Check to see whether the event list is empty. */

    if (next_event_type == 0)
    {

        /* The event list is empty, so stop the simulation */

        fprintf(outfile, "\nEvent list empty at time %f", sim_time);
        exit(1);
    }

    /* The event list is not empty, so advance the simulation clock. */

    sim_time = min_time_next_event;
}
```

DES Example – *C implementation*

```
void arrive(void)  /* Arrival event function. */
{
    float delay;

    /* Schedule next arrival. */

    time_next_event[1] = sim_time + expon(mean_interarrival);

    /* Check to see whether server is busy. */

    if (server_status == BUSY) {

        /* Server is busy, so increment number of customers in queue. */

        ++num_in_q;

        /* Check to see whether an overflow condition exists. */

        if (num_in_q > Q_LIMIT) {

            /* The queue has overflowed, so stop the simulation. */

            fprintf(outfile, "\nOverflow of the array time_arrival at");
            fprintf(outfile, " time %f", sim_time);
            exit(2);
        }

        /* There is still room in the queue, so store the time of arrival of the
           arriving customer at the (new) end of time_arrival. */

        time_arrival[num_in_q] = sim_time;
    }
```

DES Example – *C implementation*

```
    else {

        /* Server is idle, so arriving customer has a delay of zero.  (The
           following two statements are for program clarity and do not affect
           the results of the simulation.) */

        delay             = 0.0;
        total_of_delays += delay;

        /* Increment the number of customers delayed, and make server busy. */

        ++num_custs_delayed;
        server_status = BUSY;

        /* Schedule a departure (service completion). */

        time_next_event[2] = sim_time + expon(mean_service);
    }
}
```

DES Example – *C implementation*

```
void depart(void)  /* Departure event function. */
{
    int   i;
    float delay;

    /* Check to see whether the queue is empty. */

    if (num_in_q == 0) {

        /* The queue is empty so make the server idle and eliminate the
           departure (service completion) event from consideration. */

        server_status      = IDLE;
        time_next_event[2] = 1.0e+30;
    }

    else {

        /* The queue is nonempty, so decrement the number of customers in
           queue. */

        --num_in_q;

        /* Compute the delay of the customer who is beginning service and update
           the total delay accumulator. */

        delay            = sim_time - time_arrival[1];
        total_of_delays += delay;

        /* Increment the number of customers delayed, and schedule departure. */

        ++num_custs_delayed;
        time_next_event[2] = sim_time + expon(mean_service);

        /* Move each customer in queue (if any) up one place. */

        for (i = 1; i <= num_in_q; ++i)
            time_arrival[i] = time_arrival[i + 1];
    }
}
```

DES Example – *C implementation*

```
void report(void)  /* Report generator function. */
{
    /* Compute and write estimates of desired measures of performance. */

    fprintf(outfile, "\n\nAverage delay in queue%11.3f minutes\n\n",
            total_of_delays / num_custs_delayed);
    fprintf(outfile, "Average number in queue%10.3f\n\n",
            area_num_in_q / sim_time);
    fprintf(outfile, "Server utilization%15.3f\n\n",
            area_server_status / sim_time);
    fprintf(outfile, "Number of delays completed%7d",
            num_custs_delayed);
}


void update_time_avg_stats(void)  /* Update area accumulators for time-average
                                     statistics. */
{
    float time_since_last_event;

    /* Compute time since last event, and update last-event-time marker. */

    time_since_last_event = sim_time - time_last_event;
    time_last_event       = sim_time;

    /* Update area under number-in-queue function. */

    area_num_in_q      += num_in_q * time_since_last_event;

    /* Update area under server-busy indicator function. */

    area_server_status += server_status * time_since_last_event;
}


float expon(float mean)  /* Exponential variate generation function. */
{
    /* Return an exponential random variate with mean "mean". */

    return -mean * log(lcgrand(1));
}
```

linear congruential method introduced in 1951. Uses the following recursive relation:

$x(k+1) = [a*x(k) + c] \bmod m$

Inverse transformation

The exponential number is obtained by using the ***inverse transformation technique***.

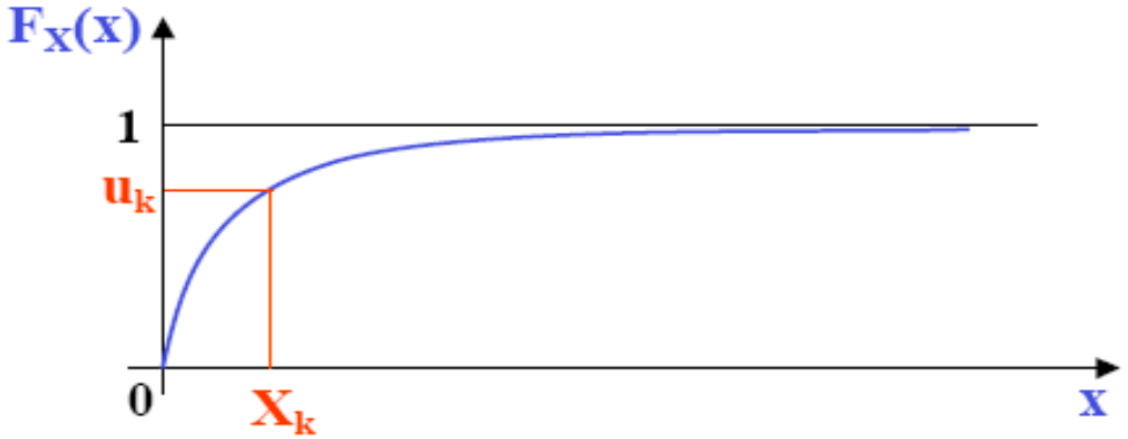
The CDF of a random variable x is defined as $F_x(x) = \int_{-\infty}^{x} x(t)dt$. It maps any value x to a probability in (0, 1).

Given u, Uniform[0,1), then the random variable: $x = F_x^{-1}(u)$ follows exactly the distribution defined by F_x

For the exponential distribution $Fx(x) = 1 - e^{-\lambda x}$, $u = F_x(x)$is a random number in the interval [0, 1). Solving for x:

$x = F_x^{-1}(u) = -\ln(1-u)/\lambda$, which is equivalent to $F_x^{-1}(u) = -\frac{\ln(u)}{\lambda}$

where λ = 1 / mean_interarrival_time



A **uniform** pseudo-random number u from the LCG is transformed into an **exponentially** distributed interarrival time x with a logarithm.